

Advanced Audio Processing: Exercise 04

Audio Classification with data augmentation

1 Introduction

In the previous exercises you have created deep neural networks (DNNs) to do classification and detection. The performance obtained with the previous exercise was not the best one, classification and detection can be improved using different techniques. Some of these techniques are transfer learning, data augmentation and scheduling the learning rate. In this exercise you will focus on techniques related to data augmentation for audio data. It is important to note that data augmentation must be applied only on the training data, since the accuracy of our model should be measured on real data.

You will use the DNN model from previous exercises and implement different data augmentation techniques and evaluate the performance of the system. In all tasks, you are free to use either a CPU or a graphics processing unit (GPU) for implementing the necessary calculations.

2 Data augmentation (1.5 points)

When the model is trained with a very small set of samples per class, it tends to “memorize” the training set rather than generalize, leading to overfitting. A group of most popular augmentation techniques will be covered in this exercise, applying these methods to the set of training data, will result in a bigger dataset. Use the template **data_augmentation_template.py** to implement the augmentation methods.

2.1 Basic 1-D alterations (0.5 point)

The most simple methods for data augmentation are adopted from computer vision. Here we will try to implement the most popular ones, such as adding noise and pitch shifting of the signal.

- Adding noise of your choice: white noise, brownian noise or even “real” noise extracted from recorded soundscapes labeled as background noises (place it in a function called **add_noise()**)
- Pitch shift [5]: the audio will sound higher or lower by shifting upwards or downwards the frequency components by a constant factor (place it in a function called **add_pitch_shift()**).

2.2 Reverberation (0.5 points)

A bit more complex technique is to simulate a sound being played in a different environment by using the room impulse response (RIR). In order to get this effect, the signal and the RIR from the desired environment are convolved, creating a new synthetically generated signal.

Specifically, you have to:

1. Extract the room impulse responses from the **.zip** file. If you wish, you can use additional RIRs of your choice.
2. Implement a function called **add_impulse_response()** that convolves the original audio file with the room impulse response.

2.3 specAugment (0.5 points)

The main difference from previous augmentation methods is that this technique is applied to the spectrogram, not to the raw signal.

Horizontal flip or different rotations are commonly used in image processing with good results, however the same do not apply to spectrograms, since it will substantially modify the sound that it represents. Instead SpecAugment is used, where different sections of the spectrogram are masked out. As introduced in [2] specAugment consist of three steps:

1. Time wrapping
2. Frequency mask: where randomly consecutive frequencies are masked out.
3. Time mask: where random ranges of time are masked out.

The process of masking out is performed by adding horizontal (frequency mask) or vertical (time mask) bars on the spectrogram, read [2] for more detailed information.

For this exercise, we will be implementing **Frequency Masking** only for simplicity. Specifically, you have to:

1. Randomly select a range of frequency channels that will be masked in the spectrogram.
2. The selected range is then zero-out, mapped to zeroes.

When selecting the range $[f_0, f_0+f]$, f value can be randomly sampled from a uniform distribution from 0 to the frequency max parameter (upper level, e.g. 40). The parameter f_0 is randomly selected within the range $[0, mF - f]$ where mF is the number of mel frequency channels. Place your code in a function called **freq_masking()**.

The corresponding code of all functions should be placed in the file **data_augmentation_<your name>.py**, which you will submit. Any text answers (i.e. not code) should be as comments in the Python file.

3 Dataset class (0.5 point)

Up to now you have been pre-computing the input features of the network (in some cases we provided them already) and you loaded them directly in your Dataset class. Since we want to be able to perform some augmentations on our data and some of those augmentations have to be done in time domain, in this exercise your Dataset class will load the audio files, apply the desired augmentation, and compute the input features for the network as they are needed during the training. This way, we can apply a different random transformation every time we load a file and the data in every epoch will be different.

The data used for this exercise is from ESC-50 [1], we have selected a subset from it, specifically four classes: *rain*, *sea_waves*, *chainsaw* and *helicopter*. The splits are made so the available training and validation data are reduced. To use the data, you will first have to download the data from the Moodle page of the exercise. The provided data are split in three directories, **training**, **testing** and **validation** with the corresponding csv files (train_meta.csv, test_meta.csv and val_meta.csv), which contain two columns: filename and label. The csv files can be read by using the function “read_csv” from Pandas [6].

You can use the template **dataset_class_template.py** to define your Dataset class. When loading an item, you will need to:

1. Read the corresponding audio file
2. Apply the time domain augmentations:
 - a. For the pitch shift, you must choose a random number of steps in the interval `[-pitch_shift_max_steps, +pitch_shift_max_steps]`. Note that librosa accepts a fractional number of steps, so you can use a float value.
 - b. For the reverberation, you must use a random impulse response from the provided ones.
3. Extract mel band energies, you can use the function given in **utils.py**
4. Get the label from the dataframe. Be aware that the order of the audio files in the csv files might not be the same as the one followed when reading the files from the data directories.
5. Encode the label using one-hot encoding, you can use the function given in **utils.py**, `create_one_hot_encoding`, this will encode the natural language into numbers. One-hot encoding is commonly used as input to machine learning algorithms to encode categorical data to integer data

The corresponding code should be placed in the file **dataset_class_<your name>.py**, which you will submit.

4. Train the model (1 point)

The main goal of this exercise is to improve the performance of an audio classification model by using data augmentation techniques. The model used for training will be the one implemented in exercise 2 with one main change: we will perform an average pooling along the time axis between the convolutional blocks and the final classifier layer. We can do this because our input signals are stationary; therefore, we can expect the output of the convolutional blocks to be similar across time. By pooling the time dimension, we reduce the size of the fully-connected classifier layer, which is prone to overfitting. You can use the implementation of the model available in **my_cnn_system.py**.

A template **training_template.py** is provided, where the following steps are indicated and need to be completed:

1. Implement the training, validation, and testing processes of your system, you can use your own code or the provided code from the previous exercise(s), which you're allowed to modify in any way that suits you.
2. Use one or combination of augmentation techniques for training your model.

3. Set-up the loop over epochs.
4. Set-up the early stopping process.
5. Log training, validation, and testing losses and print them to console (where appropriate).
6. Since the problem is multi-class classification you should use the cross-entropy loss. A part of calculating the loss, for better understanding of the output, calculate the accuracy of the model and represent the confusion matrix [4] (CM) for the testing data. The CM helps to identify which classes are more difficult to classify, giving a more detailed analysis of the model performance.
7. Compare the model performance using data augmentation techniques on the training data against when only the original training samples are used.

The corresponding code should be placed in the file **training_<your name>.py**, which you will submit.

Write a short report (around 1 page) about the results. Report the performance of the trained models and reflect on the following questions:

- Which of the used augmentation techniques has helped (or not) to increase the model accuracy?
- Was it only one method or did you use a combination of them? Comment on this.

You should submit all your code (even the code that you have developed in previous exercises), including **training_<your name>.py**, **data_augmentation_<your name>.py**, **dataset_class_<your name>.py**, and any additional source code that you created or modified, and a PDF report **report_ex4_<your name>.pdf**. Place all the files in a zip archive **ex4_<your name>.zip**.

References

- [1] <https://github.com/karolpiczak/ESC-50>
- [2] <https://ai.googleblog.com/2019/04/specaugment-new-data-augmentation.html>
- [4] https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html
- [5] <https://librosa.org/doc/main/effects.html>
- [6] https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html